

C# Programming Lab

Hospital Patient System

OOP • Inheritance • Abstract Classes • Polymorphism

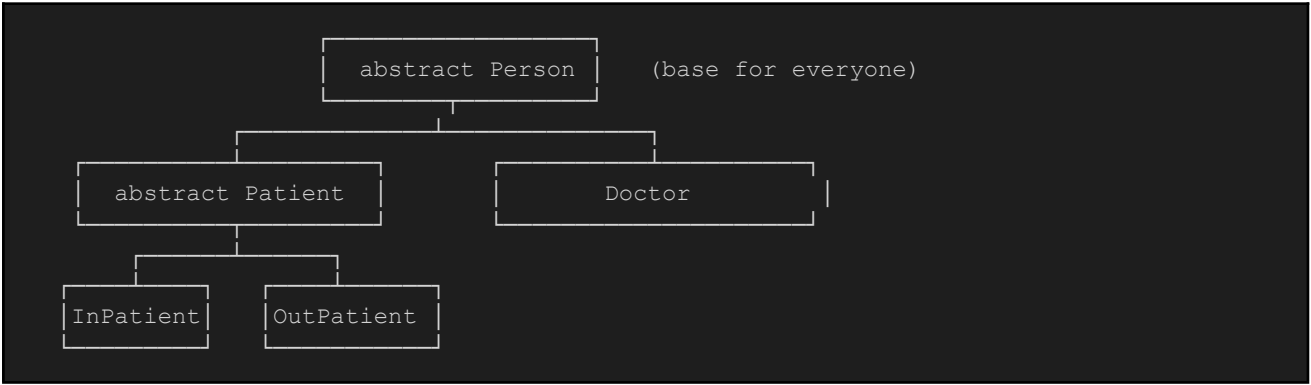
Lab Title	Hospital Patient System
Language	C# (.NET Console Application)
Concepts Covered	Abstract Class, Inheritance, Polymorphism, Method Overriding, Enum, List<T>

Objective

Design and build a hospital management system that models different types of people using an inheritance hierarchy. You will create abstract base classes, concrete subclasses, and demonstrate polymorphism by storing different types in a single list and calling overridden methods on them.

Class Hierarchy - Overview

Build exactly this inheritance tree. Do not change class names or relationships.



Class Definitions

PatientStatus (Enum)

```
enum PatientStatus
{
    Admitted,
    OutPatient,
    Discharged
}
```

abstract class Person

The root of the hierarchy. Every person in the system - patient or doctor - is a Person.

```
abstract class Person
{
    public string Name { get; set; }
    public string ID   { get; set; }    // e.g. "P001", "D001"

    public Person(string name, string id)
    {
        Name = name;
        ID   = id;
    }

    public abstract string GetRole();
    public abstract void  DisplayInfo();
}
```

abstract class Patient : Person

Extends Person. Still abstract - cannot be instantiated directly.

```
abstract class Patient : Person
{
    public string      Diagnosis { get; set; }
    public PatientStatus Status   { get; set; }

    public Patient(string name, string id, string diagnosis)
        : base(name, id)
    {
        Diagnosis = diagnosis;
        Status     = PatientStatus.Admitted;    // default
    }

    public abstract double CalculateBill();
}
```

class InPatient : Patient

A patient who stays in the hospital. Bill is calculated from daily rate × days.

```
class InPatient : Patient
{
    public int    RoomNumber { get; set; }
    public int    DaysStayed { get; set; }
    public double DailyRate  { get; set; }

    public InPatient(string name, string id, string diagnosis,
                     int roomNumber, double dailyRate)
        : base(name, id, diagnosis)
    {
        RoomNumber = roomNumber;
        DailyRate  = dailyRate;
        DaysStayed = 0;
    }

    public override string GetRole()           => "InPatient";
    public override double CalculateBill()     => DaysStayed * DailyRate;
    public override void   DisplayInfo()       { /* Part 2 */ }
}
```

class OutPatient : Patient

A patient who visits for an appointment. Bill is a fixed consultation fee.

```
class OutPatient : Patient
{
    public string AppointmentDate { get; set; } // e.g. "20260915"
    public double ConsultationFee { get; set; }

    public OutPatient(string name, string id, string diagnosis,
                     string appointmentDate, double consultationFee)
        : base(name, id, diagnosis)
    {
        AppointmentDate = appointmentDate;
        ConsultationFee = consultationFee;
        Status          = PatientStatus.OutPatient;
    }

    public override string GetRole()           => "OutPatient";
    public override double CalculateBill()     => ConsultationFee;
    public override void   DisplayInfo()       { /* Part 2 */ }
}
```

class Doctor : Person

A doctor who is not a patient. Manages a list of patients under their care.

```
class Doctor : Person
{
    public string      Specialty { get; set; }
    public List<Patient> Patients { get; set; }

    public Doctor(string name, string id, string specialty)
        : base(name, id)
    {
        Specialty = specialty;
        Patients  = new List<Patient>();
    }

    public override string GetRole()      => "Doctor";
    public override void  DisplayInfo()  { /* Part 2 */ }
}
```

Requirements

Part 1 - DisplayInfo() for each class

Implement DisplayInfo() in InPatient, OutPatient, and Doctor. Each must print its own relevant fields in a clean, labeled format.

InPatient output example:

```
[InPatient]  Ahmed Samy   (P001)

Diagnosis    : Appendicitis
Status       : Admitted
Room         : 204
Days Stayed  : 3
Daily Rate   : $150.00
Total Bill   : $450.00
```

OutPatient output example:

```
[OutPatient] Mona Khaled  (P002)

Diagnosis    : Migraine
Status       : OutPatient
Appointment   : 20260915
Consult Fee   : $75.00
```

Doctor output example:

```
[Doctor]  Dr. Sara Nour  (D001)

Specialty   : Cardiology
Patients    : 3
```

Part 2 - Hospital Operations

In Main(), declare these two lists and implement the following operations:

```
List<Patient> patients = new List<Patient>();
List<Doctor>  doctors  = new List<Doctor>();
```

2a. AdmitPatient()

```
static void AdmitPatient(List<Patient> patients)
```

- Ask: type of patient (1 = InPatient, 2 = OutPatient).
- Collect the required fields based on type.
- Create the correct object and add it to patients.
- Print: "Patient admitted: {name}"

2b. DischargePatient()

```
static void DischargePatient(List<Patient> patients, string patientId)
```

- Search for the patient by ID.
- If not found → print error.
- If already Discharged → print "Patient already discharged."
- Otherwise → set Status = Discharged and print confirmation with the final bill.

2c. AssignDoctor()

```
static void AssignDoctor(List<Doctor> doctors, List<Patient> patients, string doctorId,
string patientId)
```

- Find the doctor and patient by their IDs.
- If either not found → print the appropriate error.
- If patient already assigned to this doctor → print "Already assigned."
- Otherwise → add patient to doctor's Patients list, print confirmation.

2d. DisplayAll()

```
static void DisplayAll(List<Patient> patients, List<Doctor> doctors)
```

- Call DisplayInfo() on every patient, then every doctor.
- This is pure polymorphism - one loop, one method call, different output per type.
- Show totals at the end: total patients, total doctors, total admitted.

Part 3 - Queries

3a. FilterByStatus()

```
static void FilterByStatus(List<Patient> patients, PatientStatus filter)
```

- Print all patients whose Status matches the filter.
- Show: name, ID, role (GetRole()), diagnosis.
- If none → print "No patients with status: {filter}"
- No LINQ - use a regular loop.

3b. GetTotalBill()

```
static double GetTotalBill(List<Patient> patients)
```

- Loop through all patients and sum CalculateBill().
- Return the total. In Main(), print: "Total Outstanding Bills: \${total:N2}"
- Polymorphism in action - same method call, different calculation per type.

3c. GetDoctorReport()

```
static void GetDoctorReport(List<Doctor> doctors)
```

- For each doctor, print their info and list all assigned patients (name + status).
- If a doctor has no patients → print "No patients assigned yet."
- Print the doctor with the most patients at the end.

Hints

Calling an abstract method polymorphically:

```
// patients holds InPatient and OutPatient objects
foreach (Patient p in patients)
{
    p.DisplayInfo();           // calls the correct override automatically
    p.CalculateBill();         // InPatient or OutPatient version - C# decides
}
```

Search by ID in a list:

```
Patient found = null;
foreach (Patient p in patients)
    if (p.ID == patientId)
        { found = p; break; }

if (found == null) Console.WriteLine("Patient not found.");
```

Check if patient already assigned to doctor:

```
bool alreadyAssigned = false;
foreach (Patient p in doctor.Patients)
    if (p.ID == patient.ID)
        { alreadyAssigned = true; break; }
```

base constructor call:

```
class InPatient : Patient
{
    public InPatient(...) : base(name, id, diagnosis)    // calls Patient's constructor
    { ... }
}
```

Main Menu

Run a dowhile loop with these options:

```
                HOSPITAL MANAGEMENT SYSTEM

1. Admit Patient
2. Discharge Patient
3. Assign Doctor to Patient
4. Display All
5. Filter Patients by Status
6. Total Outstanding Bills
7. Doctor Report
8. Exit
```

Bonus Challenge

UpdateDaysStayed()

```
static void UpdateDaysStayed(List<Patient> patients, string patientId, int days)
```

- Find the patient by ID.

- If they are NOT an InPatient → print "Only inpatients have days stayed."
- Otherwise → update DaysStayed and print the new bill.
- [object Object],[object Object],[object Object]

Add class EmergencyPatient : InPatient

- Adds a field: EmergencyFee (double) = 500 fixed.
- Override CalculateBill(): return base.CalculateBill() + EmergencyFee.
- Override GetRole(): return "Emergency".
- Override DisplayInfo() to also show the emergency fee.
- Add it as a 3rd option when admitting a patient.
- It should work seamlessly inside the existing patients list - no other code changes needed.

Good luck!